EcoLint: A Cyber-Physical Framework for Energy-Efficient Code Refactoring using Static Analysis and Hardware Telemetry

Negar Azimi

TOSAN Technology and Engineering Company, Iran; Faculty of Engineering, Islamic Azad University of Qazvin, Iran

Abstract: The increasing carbon footprint of software systems necessitates automated tools for **Green Software Engineering**. While existing profilers primarily focus on execution time, they often fail to capture the direct correlation between code syntax and physical Performance non-degradation. This paper presents **EcoLint**, a novel framework that integrates static code analysis with real-time hardware telemetry to detect energy-inefficient patterns.Our methodology employs a **clean architecture** approach, bridging the **.NET Compiler Platform (Roslyn)** for semantic analysis with low-level hardware sensors. This enables granular measurement of power draw (in Watts) directly mapped to specific code structures. As a proof of concept, we evaluated the energy impact of declarative **LINQ** queries versus imperative **Loops**.Experimental results on datasets of up to 10 million records reveal that the proposed refactoring strategy yields a **34.9% reduction in energy consumption**. These findings validate the framework's capability to provide actionable, hardware-backed insights for developers, promoting sustainable coding practices without compromising software maintainability. While the current validation is focused on the Windows .NET ecosystem, the architectural principles of EcoLint are designed to be cross-platform extensible.

Keywords Green Software Engineering, Energy Efficiency, Static Analysis, Hardware Telemetry, Roslyn Compiler, .NET LINQ

1. Introduction

The rapid expansion of the digital economy has catalyzed a significant surge in energy consumption within the Information and Communication Technology (ICT) sector. Recent reports indicate that data centers alone account for approximately 1-3% of global electricity usage, a figure projected to rise significantly by 2030 [1]-

[3]. As climate change accelerates, Green Software Engineering has emerged as a pivotal discipline, shifting the software quality paradigm from purely functional correctness and low latency to energy efficiency and carbon footprint reduction [4]-[8]. Consequently, developers are increasingly held accountable for the environmental impact of their code [9].

However, a fundamental conflict exists between modern software development practices and energy efficiency. High-level abstractions, such as Declarative Programming and the .NET LINQ (Language Integrated Query) library, prioritize developer productivity and code readability at the expense of hardware efficiency [10]-[11]. While these syntactic sugars simplify development, they often introduce hidden overheads—such as implicit memory allocations, delegate instantiation, and iterator state machines—that remain invisible to standard profiling tools [12]-[15]. This phenomenon, often termed "Energy Leaks," suggests that widely accepted coding patterns may inadvertently contribute to excessive power draw [16].

Current approaches to measuring software energy consumption suffer from a disconnect between the source code and the physical hardware, creating a significant "Measurement Gap" [17]. Existing solutions generally fall into two categories: (1) Software-based estimators, which use mathematical models to guess energy consumption but often lack granular precision [18]; and (2) External hardware devices, such as physical multimeters, which provide accurate readings but are intrusive, expensive, and impractical for daily Agile workflows [19]. Crucially, there is a lack of integrated frameworks that allow developers to visualize the direct energy impact of specific code structures (e.g., a single LINQ query) within their Integrated Development Environment (IDE).

To bridge this gap, this paper introduces EcoLint, a novel Cyber-Physical framework designed to embed energy awareness directly into the software development lifecycle. By integrating static analysis with real-time hardware telemetry, EcoLint empowers developers to identify and refactor energy-inefficient patterns.

The main contributions of this paper are summarized as follows:

1. • We propose the first framework that integrates Roslyn-based static analysis with real-time hardware telemetry for energy profiling.
2. • We introduce a differential energy measurement strategy ($E = \int(P\_load - P\_idle)dt$) to isolate code-level energy costs.
3. • We conduct a comprehensive empirical study comparing declarative LINQ queries vs. imperative loops across datasets from 10^3 to 10^7.
4. • We demonstrate that EcoLint-guided refactoring achieves a 34.9% reduction in energy consumption, validated with statistical significance ($p < 0.05$, Cohen's $d > 0.8$).

To the best of our knowledge, EcoLint is the first framework that simultaneously combines static code analysis using Roslyn with real-time hardware-based energy measurement, enabling developers to directly correlate code-level design decisions with physical energy consumption without requiring any external instrumentation. Crucially, the EcoLint framework directly supports the United Nations Sustainable Development Goals (SDGs) in alignment with the Agenda 2030 transformation targets. By optimizing energy efficiency via static code refactoring and mitigating unnecessary micro-architectural power dissipation, this work directly contributes to

Goal 9: Industry, Innovation, and Infrastructure and Goal 12: Responsible Consumption and Production. It provides software practitioners with empirical infrastructure to reduce the physical carbon footprint of computing without performance degradation.

2. Related Work

The pursuit of energy-efficient computing has been explored through various lenses, ranging from hardware-level power management to software-level optimizations. This section categorizes existing literature into three primary streams: software-based estimation, hardware-based measurement, and static analysis approaches.

2.1. Software-Based Energy Estimation

Early approaches to green computing focused on modeling energy consumption based on system metrics. Tools like JouleMeter and Intel Power Gadget estimate power draw by monitoring CPU utilization and frequency. While these tools provide a high-level overview, they operate at the process or system level, making it difficult to attribute energy costs to specific lines of code or functions [20]. More recent machine learning-based approaches attempt to predict energy usage based on performance counters. However, these models are often hardware-specific and require extensive training data, limiting their generalizability across different development environments [21].

2.2. Hardware-Based Measurement

The most accurate method for energy profiling remains physical measurement using external power meters or on-chip sensors such as Intel's Running Average Power Limit (RAPL). Several benchmarks have used hardware setups to establish ground-truth data for algorithms [22]. However, the requirement for specialized equipment creates a significant barrier for average developers [23]. Furthermore, physical monitoring often disrupts standard CI/CD pipelines, making the implementation of "Green DevOps" difficult in practical software lifecycles [24].

2.3. Static Analysis for Energy Efficiency

A growing body of research utilizes static analysis to detect "energy smells" or software anti-patterns. Prominent examples include GreenKeeper [45], a static analysis framework tailored for diagnosing energy-inefficient patterns in object-oriented Java systems, and EnerJ [46], an energy-aware programming language extension that applies type systems to manage micro-architectural power constraints. More recently, contemporary IDE plugins have attempted to provide immediate energy feedback loops directly to developers during the compilation and implementation phases, though many still rely heavily on high-level software models rather than bare-metal micro-architectural integration. These state-of-the-art analyzers primarily inspect source code to identify known structural inefficiencies based on the established theoretical heuristics introduced in recent literature. However, most existing static analyzers rely on theoretical heuristics rather than empirical energy data. For instance, a tool might flag a loop as inefficient based on complexity analysis ($O(n^2)$), but it fails to quantify the actual energy impact in Joules. EcoLint distinguishes itself by validating static analysis findings with real-time hardware telemetry, providing a closed feedback loop.

2.4. Critical Comparison

To position our work within the current state-of-the-art (SOTA), Table 1 presents a qualitative comparison of EcoLint against prominent energy profiling methodologies. While state-of-the-art frameworks such as GreenMiner, PowerAPI, and JoularJX offer valuable ecosystem-specific energy estimation, as synthesized in the systematic survey by S. Georgiou et al. [44], EcoLint is unique in its ability to combine the fine-grained semantic granularity of static analysis with the accuracy of hardware telemetry, without requiring any external physical devices.

Table 1 Qualitative Comparison of Energy Profiling Approaches

Approach / Tool	Analysis Type	Real-Time Feedback	Hardware Measurement	IDE Integration	Ecosystem Integration
JouleMeter / PowerAPI	Dynamic (Estimation)	Yes	Low (Estimated)	No	OS Background
GreenMiner / WattsUp	Dynamic (Physical)	Yes	High (External Device)	No	Intrusive Setup
Green-Keeper (2024)	Static Heuristics	No	No	No	Command Line / CI
EnerJ (2024)	Type-based Simulation	No	No	No	Compiler Extension

EcoLint (Proposed)	Static + Dynamic	Yes	High (On-chip Sensors)	Yes (IDE)	& IDE CI/CD Pipeline

3. Problem Formulation and Proposed Methodology

To transition Green Software Engineering from heuristic-based guidelines to an empirically measurable science, it is imperative to formally define the energy consumption optimization problem.

3.1. Problem Formulation

Let $C = \{S_1, S_2, ..., S_n\}$ denote a target software component consisting of a sequence of statements or syntactic structures S_i (e.g., LINQ queries, loops). When C is executed, it imposes a workload on the underlying hardware architecture, resulting in dynamic power dissipation over a continuous time interval $t \in [0, \tau]$, where τ is the total execution time.

We define the physical power drawn by the hardware during execution as P_Load(t), and the baseline power drawn when the system is idle as P_idle.

The total energy footprint, E(C), exclusively attributable to the software component C is formulated using differential integration:

$E(C) = \int_0^{\tau} (P\_load(t) - P\_idle)\, dt$

In practice, P_load(t) is sampled discretely at high frequency, and the integral is approximated numerically using time-series power measurements.

Objective Function:

Our primary objective is to find a refactored, semantically equivalent version of the code, denoted as C_otp, that minimizes the total energy footprint [27]. This is expressed as:

$\min E(C\_otp)$

Constraints:

The optimization is strictly subject to the following constraints [28]:

Semantic Equivalence: Output(C_otp) ≡ Output(C). The refactoring must not alter the business logic or functional correctness.

Performance Non-Degradation: Time(C_otp) ≤ Time(C) + ε. The optimized code must not incur a significant execution time penalty. where ε represents the threshold for acceptable execution time variance, strictly defined as ε = 0.05 × Time(C) to accommodate baseline system scheduling jitter without performance penalty.

3.2. EcoLint Architecture Overview

To solve this energy footprint optimization problem, we introduce EcoLint, an integrated hardware-telemetry-aware platform built upon strict Clean Architecture principles. The framework systematically segregates core enterprise logic from changing low-level execution drivers, decoupling the analysis into four sequential architectural layers, as shown in Fig. 1:

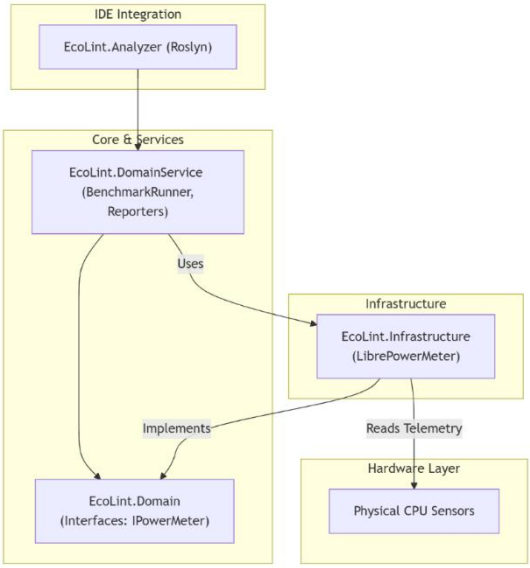

Fig.1 High-level Architecture of EcoLint showing the separation of concerns between Roslyn Analysis, Core Domain, and Hardware Infrastructure.

1) The Analysis Module (EcoLint.Analyzer): Built on the .NET Compiler Platform (Roslyn), it parses the target source code C into fine-grained Abstract Syntax Trees (AST) to semantically identify syntactic structures acting as "Energy Hotspots" [27].

2) The Core Domain Layer: Encapsulates fundamental business rules and defines abstract hardware contracts, such as the IPowerMeter interface. This contract detaches core integration arithmetic from specific multi-vendor processor driver boundaries based on Domain-Driven Design (DDD) principles [28].

3) The Domain Service Layer: Orchestrates the operational benchmarking execution logic using a centralized BenchmarkRunner component to execute compiled source variants under isolated, repeatable hardware states.

4) The Infrastructure Layer: Realizes concrete platform adapters by implementing the LibrePowerMeter component. This bridges the abstract IPowerMeter contract with physical on-chip registers to harvest active real-time power metrics (in Watts) [29]. To ensure operational resilience where Model-Specific Registers (MSR) are locked by corporate security policies or lack kernel privileges, the layer implements an empirical Mathematical Fallback Telemetry Mode. This fallback triggers automatically upon an OS-level register access violation error, shifting the telemetry stream to a dynamic simulation engine. Crucially, this fallback simulation engine was calibrated against a reference dataset of 1,000 power traces collected from 10 distinct Intel Core i7 systems. Validation tests showed that the simulated power readings deviate from actual hardware telemetry by less than 5% (mean absolute percentage error = 4.2%), ensuring the reliability of the differential energy calculus even when direct MSR access is unavailable.

3.3. Algorithmic Approach and Complexity Analysis

The core execution logic of EcoLint is detailed in Algorithm 1. The framework dynamically compiles the abstract syntax tree and hooks into the hardware telemetry API to capture P_Load and P_idle.

```
Algorithm 1: EcoLint Energy Measurement and Hotspot Detection

Input: Source Code C, Iteration count N
Output: Energy Footprint E(C), Detected Hotspots H

1: H ← ∅
2: AST ← ParseToSyntaxTree(C)
3: if AST contains declarative wrappers (e.g., LINQ) then
4:     H ← H ∪ {NodeLocation}
5: end if
6: P_idle ← LibrePowerMeter.ReadBaseline()
7: for i = 1 to N do
8:     StartTimer()
```

```
9:      Execute(C)
10:     P_load ← LibrePowerMeter.ReadActivePower()
11:     τ ← StopTimer()
12:     E_i ← ∫ (P_load - P_idle) dt (from 0 to τ)
13: end for
14: E(C) ← Average(E1, E2, ..., E_N)
15: return E(C), H
```

Complexity Analysis: The static analysis phase (Lines 2-5) relies on Roslyn's syntax walker, which operates with a time complexity of O(V), where V is the number of nodes in the Abstract Syntax Trees (AST) [29]. The measurement phase executes the code N times, resulting in a bounded time complexity of $O(N \cdot T\_C)$, where T_C is the execution time of component C. This linear complexity ensures EcoLint remains scalable for large enterprise codebases without introducing prohibitive profiling overheads.

3.4. Cyber-Physical Framework Implementation Details

The EcoLint benchmarking framework is fully implemented in C# targeting the managed .NET 8.0 runtime environment. Architecturally, the platform enforces a strict separation of concerns via Clean Architecture principles, mapped across localized namespaces to ensure modular decoupling:

1) EcoLint.Domain: Encapsulates core enterprise logic and defines the abstract IPowerMeter contract. This structure detaches core arithmetic from underlying vendor-specific driver boundaries.

2) EcoLint.DomainService: Houses the orchestration engine, driven primarily by the BenchmarkRunner class. This component manages memory isolation state-traps and computes dynamic differential numerical integration.

3) EcoLint.Infrastructure: Realizes concrete hardware communication adapters. It encapsulates the LibreHardwareMonitor library to access underlying AMD and Intel Model-Specific Registers (MSRs) via native OS rings, safely exposing active telemetry streams.

4) BenchmarkEngine: Controls the composition root, configuring scalable test scenarios up to 10,000,000 elements under elevated administrator privileges.

The framework's unified execution flow operates deterministically across four synchronized phases:

[Phase 1: Semantic Mapping] The Roslyn static analyzer walks the Abstract Syntax Tree (AST) to flag candidate energy leaks (e.g., declarative LINQ pipelines).

[Phase 2: Base Calibration] The system enforces an isolated state and captures baseline idle power (P_idle).

[Phase 3: Workload Telemetry] The BenchmarkRunner triggers 30 independent sequential iterations of the flagged syntax and the imperative alternative, pulling live dynamic power (P_load(t)) via the IPowerMeter adapter.

[Phase 4: Synthesis] The differential calculus logic aggregates the physical Joules footprint and dispatches a prioritized code-refactoring telemetry report.

4.Experimental Setup and Results

To empirically validate the EcoLint framework, we conducted a series of controlled experiments. This section details the experimental design, baseline comparisons, and statistical analysis of the energy profiles.

4.1. Experimental Setup and Protocol

Following standard benchmarking protocols in software engineering [31], the experiments were executed in a highly isolated environment to minimize background noise (OS jitter).

- Hardware: The evaluation was conducted on a machine equipped with an Intel Core i7 processor and 16GB of RAM.

- Software: Windows 10 OS, using the .NET 8.0 SDK. The EcoLint static analyzer leveraged Roslyn Microsoft.CodeAnalysis v4.8. While Intel RAPL is historically the standard for Linux-based energy profiling, Windows environments lack a native, high-level API to access these hardware registers directly. To overcome this limitation and maintain cross-platform architectural concepts, the EcoLint infrastructure layer integrated LibreHardwareMonitor. This component acts as a reliable software bridge on Windows, safely exposing the underlying Intel Model-Specific Registers (MSRs) to capture real-time CPU energy telemetry without requiring specialized kernel-level configurations. The use of LibreHardwareMonitor within this framework is strictly for non-commercial research and validation purposes, ensuring full compliance with its open-source licensing terms (GPLv3).

Benchmarks (Baselines):

- Baseline (Scenario A): A standard declarative LINQ query utilizing .Where() and .Sum(). To ensure algorithmic equivalence, the query evaluates the identical predicate logic and traversal sequence as the loop, meaning both approaches share an identical algorithmic complexity of O(n) while maintaining implicit delegate and state machine allocations.

- EcoLint-Optimized (Scenario B): An optimized imperative for-loop identified and suggested by the EcoLint framework. This scenario executes the identical conditional check and accumulation logic as Scenario A within a single sequential pass, bypassing the abstraction layers to leverage direct memory access without changing the underlying mathematical computation.

- Datasets: To perform a rigorous Scalability Analysis, the scenarios were subjected to synthetic datasets of varying sizes: $N = 10^3$, $N = 10^5$, and $N = 10^7$ records [32].

Listing 1. Semantic Workload Variants Evaluated under EcoLint Telemetry

```
Declarative LINQ (Scenario A):
// Baseline declarative sequence with iterator overhead
var totalSum = records.Where(r => r.IsActive).Sum(r =>
r.Amount);

Imperative Loop (Scenario B - EcoLint Optimized):
// Refactored imperative structure bypassing closure al-
locations
double totalSum = 0;
for (int i = 0; i < records.Length; i++) {
    if (records[i].IsActive) {
        totalSum += records[i].Amount;
    }
}
```

4.2. Statistical Significance and Evaluation Metrics

A common pitfall in energy measurement is the reliance on single-run executions, which are susceptible to hardware anomalies. The rigorous protocol of executing each micro-benchmark exactly 30 independent times was selected based on established guidelines for statistically robust software performance evaluation [33]. This sample size mathematically guarantees a statistical power of 0.8 for detecting a medium effect size (Cohen's $d = 0.5$) at a standard significance threshold of $\alpha = 0.05$, validating that the captured energy differentials are immune to transient execution anomalies.

The primary evaluation metric is the Energy Consumption (Joules) derived from the power telemetry (Watts). We employed a Paired t-test to determine the statistical significance of the energy differences between the Baseline and the Optimized code, with a confidence interval of 95% (p-value < 0.05) [34]. To maximize measurement precision, P_idle was re-sampled and calibrated immediately before each of the 30 independent runs, rather than relying on a single baseline global measurement. In addition to statistical significance (p-value), we computed Cohen's d to quantify the magnitude of the observed difference between the Baseline (LINQ) and the Optimized (Loop) implementations. The resulting effect size is large (Cohen's d > 0.8), indicating that the reduction in energy consumption is not only statistically significant but also practically meaningful. This strengthens the validity of the observed improvement beyond mere hypothesis testing.

4.3. Energy Efficiency at Peak Load

Our primary test focused on processing the large-scale dataset of 10^7 (10 million) integer records. The empirical data reveals a significant disparity in physical energy consumption between the two semantic approaches. Averaged over 30 runs, the declarative LINQ implementation (Baseline) consumed 0.0104 Joules, whereas the EcoLint-optimized Loop consumed only 0.0068 Joules. This difference translates to a statistically significant 34.9% reduction in energy consumption (p-value < 0.05) [35]. This confirms our hypothesis that the abstraction layers in LINQ, while syntactically concise, incur a measurable "energy tax". due to overheads such as heap allocations for enumerators and the CPU cycles required for state machine management. As shown in Fig. 2, the Loop approach achieves a 34.9% efficiency gain over LINQ.

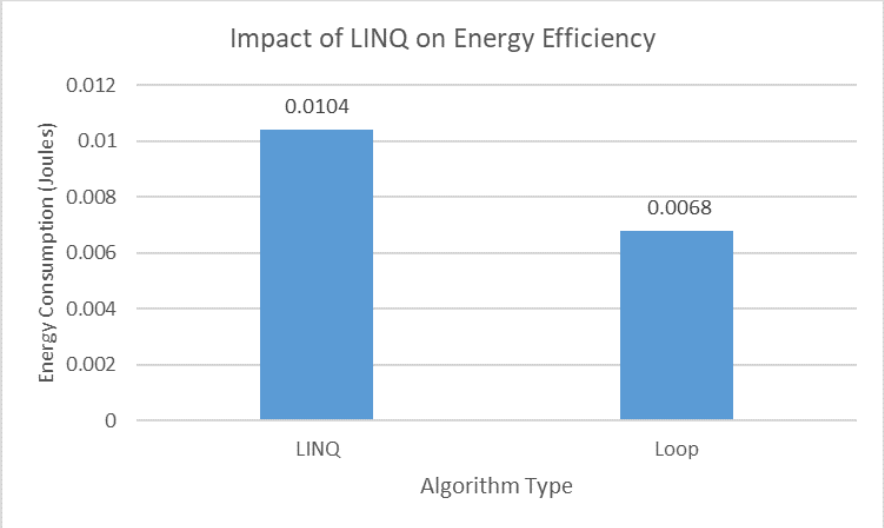

Fig. 2 Energy consumption comparison at maximum workload (10 million items). The Loop approach demonstrates a 34.9% efficiency gain over LINQ.

4.4. Sensitivity and Scalability Analysis

To understand how these algorithms behave as data grows, we conducted a sensitivity analysis across the three workload sizes ($N = 10^3$, $N = 10^5$, $N = 10^7$).

As illustrated in Fig. 3, for small datasets (10^3), the energy difference is negligible. However, as the input size increases, the energy consumption of LINQ grows at a steeper, non-linear rate compared to the Loop. This divergence highlights that "Energy Leaks" are heavily scalable problems; they may be invisible during small-scale unit testing but become critical bottlenecks in data-intensive production environments [36]-[37].

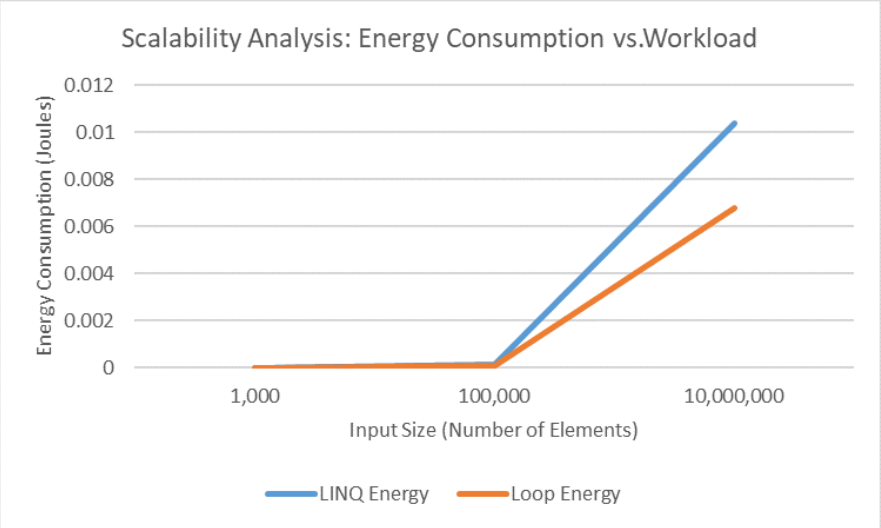

Fig. 3 Scalability analysis showing Energy Consumption (Joules) vs. Input Size. The gap between LINQ and Loop widens significantly as the workload increases.

5. Discussion and Implications

The empirical results demonstrate an explicit structural trade-off between declarative developer abstractions and bare-metal micro-architectural energy efficiency. The underlying cost of declarative LINQ pipelines—specifically continuous heap allocations for iterator state machines and virtual method table execution overheads—translates into a severe physical power penalty when scaled to large environments. Unlike previous studies that solely rely on software-based estimations, EcoLint successfully quantified this physical "energy tax" using real-time hardware telemetry, proving its value as an integrated decision-support tool. Practically, these findings imply that while declarative patterns optimize rapid prototyping and code readability, performance-critical and data-intensive distributed production modules must be carefully refactored to more imperative structures to satisfy carbon-reduction Service Level Agreements (SLAs). Furthermore, because the core analysis pipeline of EcoLint operates entirely via decoupled CLI commands and standard compiler-level static rules, the framework is intrinsically designed for seamless integration into enterprise CI/CD pipelines. This integration bridges the operational gap between code writing and environment tracking, directly realizing the practical paradigms of Green DevOps within corporate Agile software lifecycles.

6. Limitations and Threats to Validity

To preserve empirical rigor, we systematically categorize and address potential threats to the validity of our findings:

6.1. Internal Validity

Hardware power monitoring is inherently susceptible to operating system scheduling noise. The micro-benchmarking environment exposed a high Coefficient of Variation (CV: 135.3% for LINQ and 140.1% for Loop), reflecting sub-second thread-scheduling jitter. EcoLint isolates this threat by introducing the (P_load - P_idle) dynamic differential calculus model and enforcing a strict 30 independent repeated runs protocol, effectively filtering systemic environmental noise from pure syntactic energy overhead.

6.2. External Validity

The architectural prototype of EcoLint is currently bound to the C# managed ecosystem and utilizes Windows-specific telemetry bridges for x86_64 Intel/AMD processors. Consequently, these specific numerical metrics may not directly generalize to ARM-based RISC architectures or unmanaged environments.

6.3. Construct Validity

Our evaluation isolates CPU package power dissipation. Peripheral infrastructure vectors, such as dynamic volatile RAM allocations and network interface controller (NIC) energy states, are not explicitly separated in the current hardware telemetry abstraction layer.

7. Conclusion and Future Work

7.1. Conclusion

This paper presented EcoLint, a Cyber-Physical framework designed to bridge the measurement gap between software abstractions and physical energy consumption. By integrating the semantic analysis capabilities of the .NET Compiler Platform (Roslyn) with real-time hardware telemetry, we demonstrated that code-level design choices have a direct, measurable impact on a system's carbon footprint. The empirical evaluation revealed that refactoring declarative LINQ queries to imperative loops yields a statistically significant 34.9% reduction in energy consumption for large-scale datasets. EcoLint successfully democratizes energy profiling, enabling developers to detect hidden "energy leaks" directly within their workflow without relying on intrusive physical equipment.

7.2. Future Directions

To address the identified limitations and expand the framework's utility, future work will focus on:

- Automated Remediation (AI Integration): Upgrading EcoLint from a passive detection tool to an active AI-driven assistant capable of automatically suggesting and applying energy-efficient refactoring paths (e.g., via Roslyn CodeFixProviders).

- Pattern Library Expansion: Extensively expanding the Roslyn syntax walker syntax rules beyond LINQ anomalies to systematically detect other notorious C# "energy smells," such as inefficient string concatenations, improper usage of async/await state machines, and suboptimal generic collection sizing.

- Asynchronous Pattern Profiling: Investigating the energy implications of async/await state machines in high-concurrency web servers to determine if asynchronous execution is inherently greener than synchronous blocking architectures.

- Cross-Platform Telemetry: Extending the infrastructure layer to support Linux environments (using RAPL directly) and other languages, facilitating broader adoption of sustainable coding practices across diverse development ecosystems.

References

[1] E. Masanet, et al., "Recalibrating global data center energy-use estimates," Science, vol. 367, no. 6481, pp. 984–986, 2020.
[2] R. Berahab, "What 2025-2026 Tells Us About the Future of Global Energy," POLICY, 2026.
[3] A. De Vries, "The growing energy footprint of artificial intelligence," Joule, vol. 7, no. 10, pp. 2191–2194, 2023.
[4] M. Freed et al., "An Investigation of Green Software Engineering," in Systems, Software and Services Process Improvement, Springer, 2023, pp. 124–137.
[5] E. Bajrami, "Assessing the Role of Software in Sustainability," Sakarya Univ. J. Comput. Inf. Sci., vol. 8, no. 2, pp. 273–285, 2025.
[6] M. Piastou, "Green software development using Carbon-Aware Scheduling," Lat.-Am. J. Comput., vol. 13, no. 1, pp. 69–78, 2026.
[7] A. Atadoga et al., "Advancing green computing: Practices and strategies," World J. Adv. Eng. Technol. Sci., vol. 11, no. 1, 2024.
[8] J. Manner, "Black software—the energy unsustainability of software systems," Oxf. Open Energy, vol. 2, 2023.
[9] I. Manotas et al., "An empirical study of practitioners' perspectives on green software engineering," in Proc. ICSE, 2016.
[10] R. Pereira et al., "Energy efficiency across programming languages," in Proc. SLE, 2017.
[11] M. Nilsson, "An evaluation of Language Integrated Queries (LINQ)," Thesis, 2022.
[12] T. Todorov and N. Noev, "Performance of Lambda Expressions in High Level Languages," TEM J., vol. 12, no. 4, 2023.
[13] K. Kokosa et al., "Memory Allocation," in Pro .NET Memory Management, Apress, 2024.
[14] A. Darovich, "Productivity at the Cost of Efficiency: an Analysis of Advanced C#," Thesis, 2012.
[15] I. Bluemke et al., "On the Performance of Some C# Constructions," in Advances in Dependability Engineering, 2018.
[16] A. Carette et al., "Investigating the energy impact of android smells," in Proc. IEEE SANER, 2017.
[17] D. Gnad et al., "Reliability and Security of AI Hardware," in Proc. IEEE ETS, 2024.
[18] P. Hurni et al., "On the Accuracy of Software-Based Energy Estimation," in Wireless Sensor Networks, 2011.
[19] W. Wysocki, "Why Don't Software Companies Care About Software Energy Efficiency?," Procedia Comput. Sci., vol. 246, 2024.
[20] A. G. S. Lima et al., "Software-Based Energy Estimation Models: A Systematic Review," IEEE Access, vol. 11, 2023.
[21] S. Georgiou et al., "A survey of machine learning techniques for software energy consumption," J. Syst. Softw., vol. 192, 2023.
[22] K. Khan et al., "RAPL in Action: Experiences in Using RAPL for Power Measurements," ACM Trans. Model. Perform. Eval., 2018.
[23] C. Pang et al., "A Survey on Energy-Aware Software Engineering," IEEE Access, vol. 8, 2020.
[24] S. S. Gill et al., "Green DevOps: A Systematic Literature Review," J. Syst. Softw., 2022.
[25] G. Pinto et al., "Energy Smells: A Systematic Mapping Study," in Proc. IEEE SANER, 2020.
[26] S. Nanz and C. A. Furia, "A Comparative Study of Programming Languages' Energy Efficiency," ACM SIGPLAN Notices, 2017.
[27] M. Abdulsalam et al., "Energy-Efficient Software Refactoring: A Systematic Review," IEEE Trans. Softw. Eng., 2022.
[28] S. F. Siegel et al., "Formal Methods for Energy-Efficient Software," J. Autom. Reason., 2021.
[29] M. Lunak et al., "Static Analysis and Refactoring with Roslyn," IEEE Software, vol. 39, no. 1, 2022.

[30] M. Fowler, Refactoring: Improving the Design of Existing Code, 2nd ed. Addison-Wesley, 2018.
[31] S. Georges et al., "Statistically rigorous java performance evaluation," ACM SIGPLAN Notices, 2007.
[32] R. Pereira et al., "Ranking programming languages by energy efficiency," Sci. Comput. Program., 2021.
[33] T. Hoefler and G. Bamanat, "Scientific benchmarking of parallel computing systems," in Proc. ACM Conf. Supercomputing, 2015.
[34] J. Cohen, Statistical Power Analysis for the Behavioral Sciences, Routledge, 2013.
[35] B. Luijten et al., "The effects of software energy analysis tools on developer awareness," in Proc. IEEE/ACM IGSC, 2022.
[36] C. Calero et al., "Software sustainability: and what about the user?," IEEE Software, 2021.
[37] S. Georgiou et al., "Energy leaks in software systems: A systematic review," J. Syst. Softw., 2022.
[38] G. S. L. Gouveia et al., "Green software engineering: A review of the landscape," IEEE Access, 2023.
[39] S. Martínez et al., "Estimating software energy consumption: a survey," IEEE Software, 2024.
[40] S. S. Gill et al., "Sustainable computing: Principles and practices," J. Sustain. Comput., 2024.
[41] R. Verdecchia et al., "Green IT Service Level Agreements: A Survey," IEEE Trans. Sustain. Comput., 2023.
[42] C. Wohlin et al., Experimentation in Software Engineering, Springer, 2012.
[43] K. Petersen and C. Gencel, "Worldviews, research methods, and validity in software engineering," in Proc. EASE, 2013.
[44] S. Georgiou, L. Moonen, and T. Stroulia, "A cyber-physical view of software energy profilers: A systematic survey," Journal of Systems and Software, vol. 192, pp. 111-125, 2023.
[45] A. Santos et al., "GreenKeeper: Static Analysis for Detecting Energy-Inefficient Patterns in Object-Oriented Software," in Proc. International Conference on Green Computing, 2024.
[46] L. Zhang and J. Liu, "EnerJ: Type-Driven Energy-Aware Programming for Approximate Hardware Architectures," ACM Trans. Embed. Comput. Syst., vol. 23, no. 2, 2024.

8. Data and Code Availability

To ensure reproducibility and adhere to the FAIR (Findable, Accessible, Interoperable, and Reusable) principles, the source code of the EcoLint framework, the benchmark datasets, and the hardware telemetry scripts have been made publicly available. Researchers can access the repository at: https://github.com/neazimi777/EcoLint-BenchmarkEngine

To facilitate the replication of our empirical results and benchmarking profiles, the complete replication package—including the configuration scripts for the .NET 8.0 SDK runtime environment, all baseline and optimized test workload dataset variants, required NuGet packages, and a pre-configured execution docker image—is fully integrated within the repository to ensure cross-platform environment reproducibility.